

Project: Using a ZKVM to Build a Proof-of-Solvency Protocol

Vincent Xu | yx2021 | N13337595

Objective

For my final project, I propose building a simplified proof-of-solvency protocol using a zkVM (specifically SP1) with on-chain verification through a Solidity smart contract. The objective is to demonstrate how an exchange or custodial platform could cryptographically prove its total assets cover its total liabilities without revealing the underlying account data. This fits the implementation track by combining off-chain proving, on-chain verification, and smart contract integration in a practical blockchain setting.

Architecture

The system will model a toy exchange with two private datasets: a liabilities dataset (user balances) and an assets dataset (reserve balances). These will be committed to using a hash-based or Merkle-style commitment. An SP1 program will take these private datasets as input, recompute the commitments, compute the total liabilities and assets, and generate a zero-knowledge proof that the commitments are consistent and that total assets $\geq$ total liabilities. This proof will then be submitted to and verified by a Solidity contract.

Implementation & Tech Stack

The expected technical stack is **Rust, SP1, Solidity, Foundry, and TypeScript**. The core deliverables include:

- **Data Generation:** Mock datasets for liabilities and assets to simulate exchange state.
- **zkVM Program:** The solvency-checking logic written in Rust and compiled via SP1.
- **Smart Contract:** A Solidity verifier contract to validate the proof and record valid solvency attestations on-chain.
- **Integration Scripts:** TypeScript/Foundry scripts to generate proofs, submit them on-chain, and evaluate the system.

Project Scope

To keep the project appropriately scoped for an individual project, I will focus on the core workflow of zkVM-based solvency proving. The following are explicitly out of scope:

- Multi-asset support (the protocol will use a single asset denomination).
- Live integration with real on-chain reserve wallets.
- User-level inclusion proofs and incremental state updates.

Evaluation & Benchmarks

I plan to benchmark the system by measuring:

1. Proof generation time and how it scales as the size of the liabilities dataset increases.
2. On-chain verification gas costs.
3. System behavior under invalid conditions (e.g., inconsistent commitments or liabilities exceeding assets) to ensure the smart contract correctly rejects invalid proofs.